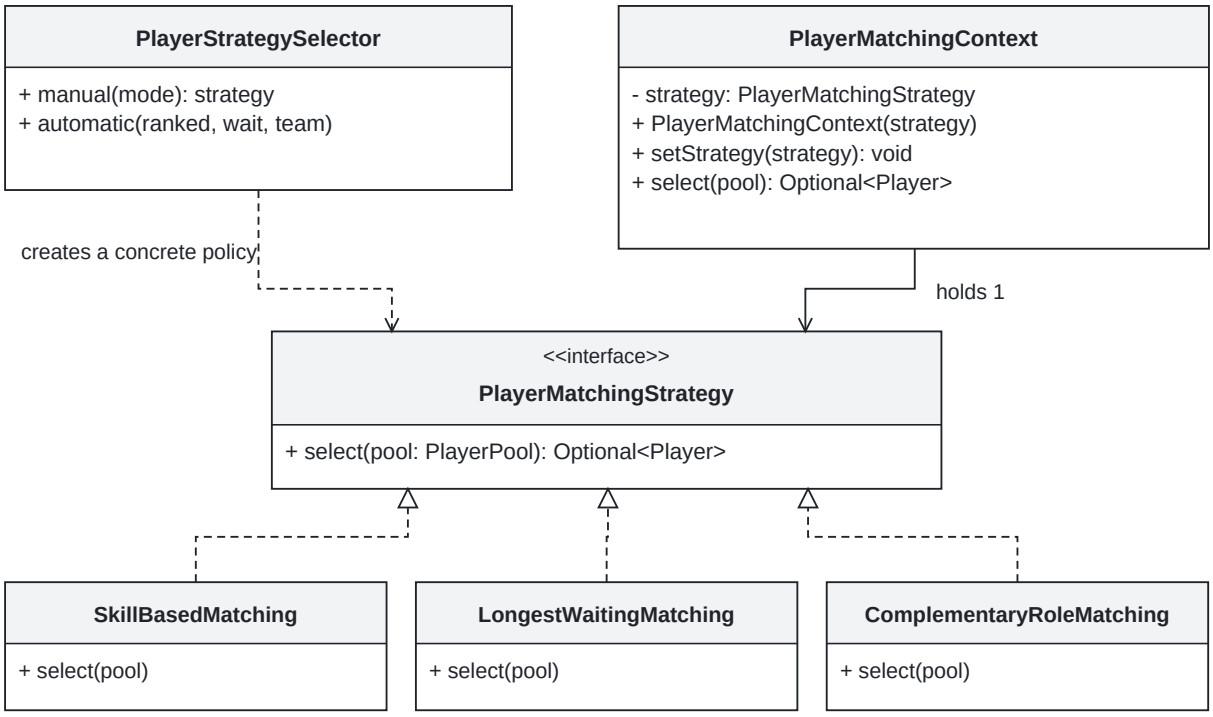
Software Design Patterns

Assignment 5

by Sabirzhanov Emil, SE-2533

Player Matching Class Diagram

The video game matchmaking project uses two Strategy families: selecting a player and selecting a server. Each family has one interface, three concrete algorithms, a replaceable context and a separate selector.



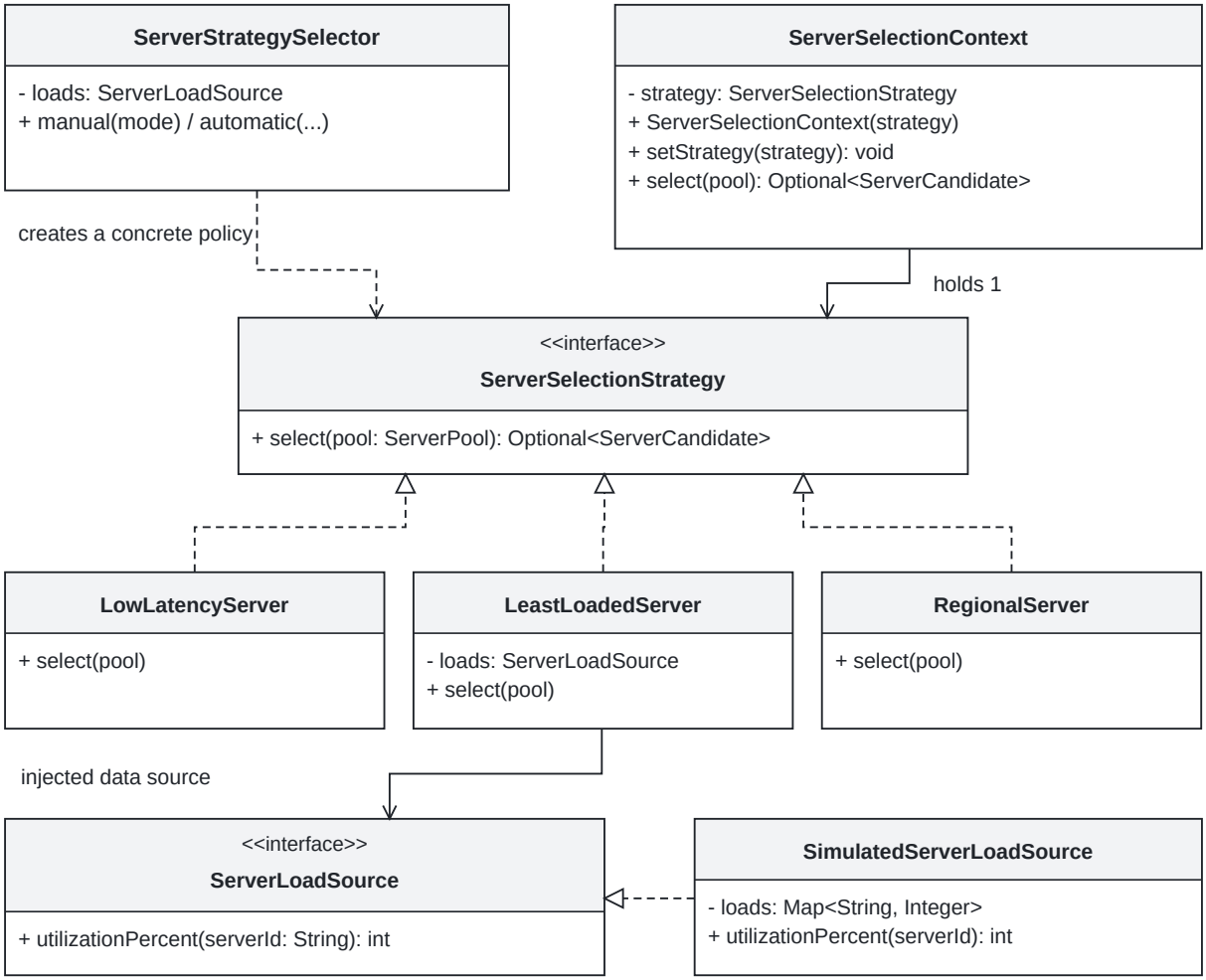
Dashed lines with hollow triangles denote interface realization. Solid arrows denote held references; dashed open arrows denote creation dependencies. UML method parameters are abbreviated outside the strategy interface.

Concrete strategy	Selection logic
SkillBasedMatching	Smallest absolute rating gap.
LongestWaitingMatching	Longest waiting time among eligible candidates.
ComplementaryRoleMatching	Different role first; smallest rating gap second.

Shared contract. `PlayerPool` excludes the seeker and rating gaps above 300. Ratings range from 0 to 5000. Ties keep the first candidate. No suitable player produces `Optional.empty()`. The algorithms select one player; they do not implement sorting algorithms.

Server Selection Class Diagram

Server policies share one selection contract but optimize different criteria. The load-aware policy obtains external measurements through our own interface.



Concrete strategy	Selection logic
LowLatencyServer	Lowest latency among eligible servers.
LeastLoadedServer	Lowest measured utilization below 100%.
RegionalServer	Preferred region first; lowest latency second.

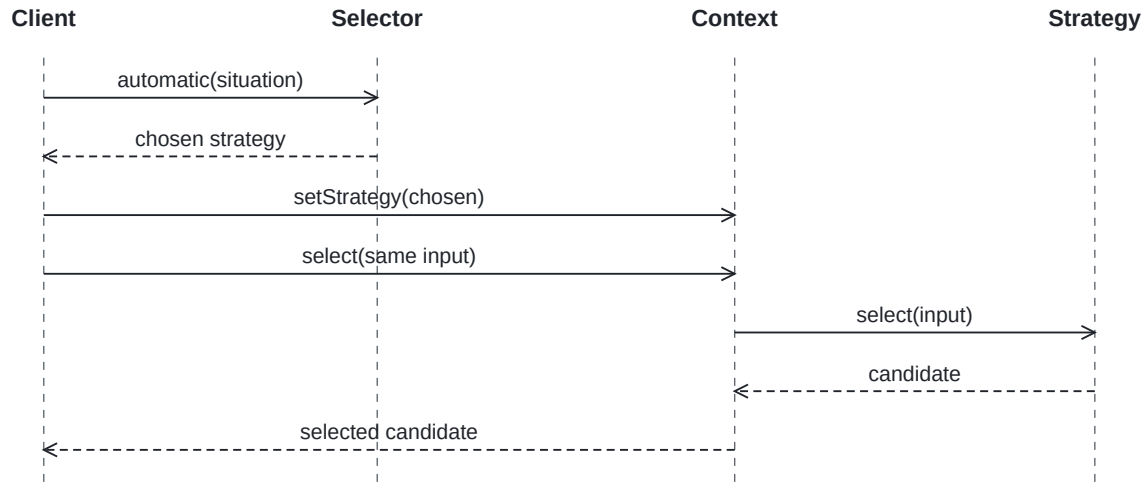
Isolation from change. `ServerLoadSource` hides the measurement source. The demo injects a simulated map; tests inject fixed fake readings. Replacing it with a monitoring adapter requires no change to the strategy or context. Each call reads every eligible server once and refreshes readings on the next call.

Shared contract. `ServerPool` excludes latency above 100 ms. Empty pools return an empty optional. Only the load policy checks utilization; full servers are excluded, unknown simulated servers count as full, invalid percentages are rejected.

Selection and Runtime Switching

Selectors evaluate rules in the following priority order. Only selectors branch on which strategy to construct; contexts simply delegate through an interface.

Family	First matching situation	Policy
Player	1. Ranked match	Skill
Player	2. Casual; seeker waited at least 120 s	Longest waiting
Player	3. Casual team match; shorter wait	Complementary role
Player	4. Remaining situations	Skill
Server	1. Competitive match	Low latency
Server	2. Noncompetitive; demand at least 80%	Least loaded
Server	3. Remaining situations	Regional



Manual selection uses `manual(Mode)`. Automatic selection uses `automatic(...)`. Both return an interface implementation. The same context accepts the returned object through `setStrategy()`.

Fixed demo inputs. Seeker: rating 1000, tank, waited 30 s. Candidates: close (1010, tank, 10 s), waiting (1200, tank, 180 s), support (1100, healer, 50 s). Preferred region: KZ. Servers: fast (EU, 20 ms, 90% load), free (US, 70 ms, 10%), local (KZ, 40 ms, 50%).

Demonstration	Player result	Server result
Same input, all algorithms	Skill: close Waiting: waiting Role: support	Latency: fast Load: free Region: local
Manual choice	Skill: close	Latency: fast
Runtime switch	close -> waiting	fast -> free
Automatic: casual team, 30 s, 90% demand	Role: support	Load: free

The automatic result enters the existing Facade with ratings 1000/1100 and latency 70 ms. The saved match reaches COMPLETED. Candidate IDs are shown in the demo; the existing CSV schema stores ratings and latency.

Design Rationale and Verification

This solution uses Strategy because each family represents interchangeable algorithms for the same operation. Player policies receive a PlayerPool and return an optional player; server policies receive a ServerPool and return an optional server. A caller can use any implementation through the same interface. Different results on identical input are expected: fairness by rating, queue waiting time and role compatibility are different matchmaking goals. Server latency, external utilization and regional preference similarly express different goals.

Each context delegates work through composition. It neither knows concrete algorithm classes nor examines their types. The client can inject a policy directly, ask a selector to choose one, or replace the policy while retaining the same context and input. To add an algorithm, implement the family interface and pass the new object to the context. Existing strategies and contexts remain unchanged. Registering that algorithm as a new built-in manual choice, or assigning it an automatic rule, is a separate change in the selector. The tests inject previously unknown implementations to demonstrate this extension point.

State also uses composition, but represents lifecycle-dependent behavior and transitions. Our policies do not describe lifecycle stages, own transitions, or replace themselves. The selector chooses a policy from the current situation, and the caller installs it. The existing NEW, RESERVED and CANCELLED party statuses belong to the earlier assignment's lifecycle model; they do not determine which matching algorithm object exists.

Template Method puts a fixed algorithm skeleton in a superclass and allows subclasses to override selected steps. Here there is no shared algorithm superclass or inherited skeleton. Each strategy owns its selection logic, and runtime replacement changes behavior through an interface reference. Shared pool filtering only enforces domain constraints; it does not call subclass hooks. This keeps the independent algorithms small and allows their data dependencies, such as server utilization, to be replaced without modifying the contexts.

Refactoring with Git History

Before: ebfded4 places all player algorithms in one ConditionalPlayerMatching switch. After: bfb4c78 extracts three strategies. PlayerStrategyTest is unchanged; all 42 tests present at that stage pass in clean snapshots of both commits.

JUnit 5 Results

Test suite	Cases	Evidence
PlayerStrategyTest	15	Three algorithms, every selector rule, boundaries, runtime replacement.
ServerStrategyTest	18	Three algorithms, every rule, fake source, refreshed readings, full servers.
StrategyDemoTest	2	Console results plus a persisted completed match through the Facade.
Existing Assignment 4 tests	27	Facade behavior, recovery, subsystem validation, console input.

62 tests passed; zero failures, errors or skipped cases. Both families have a `@ParameterizedTest` eligibility check over all three policies. Each strategy package has 100% line coverage; JaCoCo enforces at least 80% per package. Verification: 20 October 2026.

`./run-strategy.sh` runs the demo. `mvn clean verify` runs the complete test suite.

GitHub Repository: https://github.com/AlmazCode/SDP-A5

Class Responsibilities

Assignment 5 production types are listed below. Each responsibility is one sentence of at most 25 words, without the conjunctions prohibited by the assignment.

Type	Single responsibility
Player	Stores a validated queued player's attributes.
PlayerPool	Exposes eligible player candidates from an immutable snapshot.
PlayerMatchingStrategy	Defines the player selection contract.
PlayerMatchingContext	Delegates player selection to its replaceable strategy.
SkillBasedMatching	Selects the eligible player with the smallest rating gap.
LongestWaitingMatching	Selects the eligible player with the longest waiting time.
ComplementaryRoleMatching	Selects a complementary role with rating proximity as the secondary criterion.
PlayerStrategySelector	Chooses the player matching policy.
PlayerStrategySelector. Mode	Names the built-in manual player policies.
ServerCandidate	Stores a validated server candidate's attributes.
ServerPool	Exposes eligible server candidates from an immutable snapshot.
ServerSelectionStrategy	Defines the server selection contract.
ServerSelectionContext	Delegates server selection to its replaceable strategy.
LowLatencyServer	Selects the eligible server with the lowest latency.
LeastLoadedServer	Selects the least utilized eligible server using injected measurements.
RegionalServer	Selects a preferred region with latency as the secondary criterion.
ServerStrategySelector	Chooses the server selection policy.
ServerStrategySelector. Mode	Names the built-in manual server policies.
ServerLoadSource	Defines access to server utilization measurements.
SimulatedServerLoadSource	Supplies utilization measurements from a fixed snapshot.
StrategyDemo	Demonstrates interchangeable matchmaking policies through a complete match.

Live Coding Extension Points

Add a strategy by implementing the family interface and injecting it with `setStrategy()`. Change automatic behavior in the selector. Replace utilization data by supplying another `ServerLoadSource`. Existing contexts and algorithms remain unchanged when a new strategy is injected directly.